

Simple map:

```
static void example1_mapSimple() {
    System.out.println("\nExample1: Simple map");
    List<Integer> nums = Arrays.asList(1,2,3);
    List<Integer> squared = nums.stream().map(n -> n*n).collect(Collectors.toList());
    System.out.println(squared); // [1, 4, 9]
}
```

map produces nested lists:

```
static void example2_mapNestedLists() {
    System.out.println("\nExample2: Map produces nested lists");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
    List<Stream<Integer>> mapped = nested.stream().map(List::stream).toList();
    System.out.println("Mapped size: " + mapped.size()); // 2, still nested
}
```

flatMap flattens nested lists:

```
static void example3_flatMapNestedLists() {
    System.out.println("\nExample3: FlatMap flattens nested lists");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
    List<Integer> flat =
    nested.stream().flatMap(List::stream).collect(Collectors.toList());
    System.out.println(flat); // [1,2,3,4]
}
```

map strings to split arrays → nested

```
static void example4_mapStringsToChars() {
    System.out.println("\nExample4: Map strings to split arrays (nested)");
    List<String> words = Arrays.asList("hi","ok");
    List<String[]> mapped = words.stream().map(w ->
w.split("")).collect(Collectors.toList());
    System.out.println("Mapped list of arrays size: " + mapped.size()); // 2
}
```

```

static void example5_flatMapStringsToChars() {
    System.out.println("\nExample5: FlatMap strings to chars (flattened)");
    List<String> words = Arrays.asList("hi","ok");
    List<String> flat = words.stream()
        .flatMap(w -> Arrays.stream(w.split("")))
        .collect(Collectors.toList());
    System.out.println(flat); // [h,i,o,k]
}

```

```

static void example6_optionalMapVsFlatMap() {
    System.out.println("\nExample6: Optional map vs flatMap");
    Optional<String> opt = Optional.of("hello");

    Optional<Optional<Integer>> mapRes = opt.map(s -> Optional.of(s.length()));
    Optional<Integer> flatMapRes = opt.flatMap(s -> Optional.of(s.length()));

    System.out.println("Map result: " + mapRes); // Optional[Optional[5]]
    System.out.println("FlatMap result: " + flatMapRes); // Optional[5]
}

```

```

static void example7_mapToStreamNested() {
    System.out.println("\nExample7: Map to Stream (nested)");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
    Stream<Stream<Integer>> s = nested.stream().map(List::stream);
    System.out.println("Stream of streams: " + s.count()); // 2
}

```

```

static void example8_flatMapToStreamNested() {
    System.out.println("\nExample8: FlatMap to Stream (flattened)");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
    long count = nested.stream().flatMap(List::stream).count();
    System.out.println("Flattened count: " + count); // 4
}

```

```

static void example9_flatMapOptionalList() {
    System.out.println("\nExample9: FlatMap Optional containing list");
    Optional<List<Integer>> optList = Optional.of(Arrays.asList(1,2,3));
    List<Integer> flat = optList.map(List::stream).orElse(Stream.empty())
        .collect(Collectors.toList());
    System.out.println(flat); // [1,2,3]
}

```

```

static void example10_mapVsFlatMapWithEmptyOptional() {
    System.out.println("\nExample10: Empty Optional map vs flatMap");
    Optional<String> empty = Optional.empty();
    Optional<Optional<Integer>> mapRes = empty.map(s -> Optional.of(s.length()));
    Optional<Integer> flatMapRes = empty.flatMap(s -> Optional.of(s.length()));
    System.out.println("Map: " + mapRes); // Optional.empty
    System.out.println("FlatMap: " + flatMapRes); // Optional.empty
}

```

```

static void example11_flatMapMultipleStreams() {
    System.out.println("\nExample11: FlatMap multiple streams");
    Stream<String> s1 = Stream.of("a","b");
    Stream<String> s2 = Stream.of("c","d");
    List<String> combined = Stream.of(s1,s2).flatMap(s ->
s).collect(Collectors.toList());
    System.out.println(combined); // [a,b,c,d]
}

```

```

static void example12_combinedNestedCollections() {
    System.out.println("\nExample12: Combined nested collections");
    List<List<Optional<String>>> data = Arrays.asList(
        Arrays.asList(Optional.of("x"), Optional.empty()),
        Arrays.asList(Optional.of("y"))
    );
    List<String> flat = data.stream()
        .flatMap(List::stream)
        .flatMap(Optional::stream)

```

```

        .collect(Collectors.toList());
        System.out.println(flat); // [x, y]
    }

```

```

static void mapVsFlatMap1() {
    System.out.println("\nMapVsFlatMap1: Simple map (square numbers)");
    List<Integer> nums = IntStream.rangeClosed(1,
1_000_000).boxed().collect(Collectors.toList());
    long start = System.nanoTime();
    List<Integer> squared = nums.stream().map(n -> n *
n).limit(10).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("First 10 squared: " + squared + ", Time(ms): " + (end-
start)/1_000_000);
}

```

```

static void mapVsFlatMap2() {
    System.out.println("\nMapVsFlatMap2: Nested lists with map (nested)");
    List<List<Integer>> nested = new ArrayList<>();
    for(int i=0;i<1000;i++)
nested.add(IntStream.range(i*1000,(i+1)*1000).boxed().collect(Collectors.toList()));
    long start = System.nanoTime();
    List<Stream<Integer>> mapped =
nested.stream().map(List::stream).limit(2).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Mapped (Stream<Stream>) size: " + mapped.size() + ", Time(ms): "
+ (end-start)/1_000_000);
}

```

```

static void mapVsFlatMap3() {
    System.out.println("\nMapVsFlatMap3: Nested lists with flatMap (flattened)");
    List<List<Integer>> nested = new ArrayList<>();
    for(int i=0;i<1000;i++)
nested.add(IntStream.range(i*1000,(i+1)*1000).boxed().collect(Collectors.toList()));
    long start = System.nanoTime();
    List<Integer> flat =
nested.stream().flatMap(List::stream).limit(10).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flat list first 10: " + flat + ", Time(ms): " + (end-
start)/1_000_000);
}

```

```

static void mapVsFlatMap4() {
    System.out.println("\nMapVsFlatMap4: Map strings to split arrays (nested)");
    List<String> words = Arrays.asList("hi","ok","java");
    long start = System.nanoTime();
    List<String[]> mapped = words.stream().map(w-
>w.split("")).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Mapped list of arrays size: " + mapped.size() + ", Time(ms): " +
(end-start)/1_000_000);
}

```

```

static void mapVsFlatMap5() {
    System.out.println("\nMapVsFlatMap5: FlatMap strings to chars (flattened)");
    List<String> words = Arrays.asList("hi","ok","java");
    long start = System.nanoTime();
    List<String> flat = words.stream().flatMap(w-
>Arrays.stream(w.split("))).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flattened chars: " + flat + ", Time(ms): " + (end-
start)/1_000_000);
}

```

```

static void mapVsFlatMap6() {
    System.out.println("\nMapVsFlatMap6: Optional map vs flatMap");
    Optional<String> opt = Optional.of("hello");
    long start = System.nanoTime();
    Optional<Optional<Integer>> mapRes = opt.map(s->Optional.of(s.length()));
    Optional<Integer> flatMapRes = opt.flatMap(s->Optional.of(s.length()));
    long end = System.nanoTime();
    System.out.println("Map result: " + mapRes + ", FlatMap result: " + flatMapRes + ",
Time(ms): " + (end-start)/1_000_000);
}

```

```

static void mapVsFlatMap7() {
    System.out.println("\nMapVsFlatMap7: Stream of streams with map");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
    long start = System.nanoTime();

```

```

Stream<Stream<Integer>> s = nested.stream().map(List::stream);
long end = System.nanoTime();
System.out.println("Stream of streams size: 2, Time(ms): " + (end-start)/1_000_000);
}

```

```

static void mapVsFlatMap8() {
    System.out.println("\nMapVsFlatMap8: Stream of streams flattened");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
    long start = System.nanoTime();
    long count = nested.stream().flatMap(List::stream).count();
    long end = System.nanoTime();
    System.out.println("Flattened count: " + count + ", Time(ms): " + (end-
start)/1_000_000);
}

```

```

static void mapVsFlatMap9() {
    System.out.println("\nMapVsFlatMap9: Flatten Optional containing list");
    Optional<List<Integer>> optList = Optional.of(Arrays.asList(1,2,3,4,5));
    long start = System.nanoTime();
    List<Integer> flat = optList.map(List::stream).orElse(Stream.empty()).toList();
    long end = System.nanoTime();
    System.out.println("Flattened Optional: " + flat + ", Time(ms): " + (end-
start)/1_000_000);
}

```

```

static void mapVsFlatMap10() {
    System.out.println("\nMapVsFlatMap10: Empty Optional map vs flatMap");
    Optional<String> empty = Optional.empty();
    long start = System.nanoTime();
    Optional<Optional<Integer>> mapRes = empty.map(s->Optional.of(s.length()));
    Optional<Integer> flatMapRes = empty.flatMap(s->Optional.of(s.length()));
    long end = System.nanoTime();
    System.out.println("Map: " + mapRes + ", FlatMap: " + flatMapRes + ", Time(ms): " +
(end-start)/1_000_000);
}

```

```

static void mapVsFlatMap11() {
    System.out.println("\nMapVsFlatMap11: FlatMap multiple streams");
    Stream<String> s1 = Stream.of("a","b");
    Stream<String> s2 = Stream.of("c","d");
}

```

```

    long start = System.nanoTime();
    List<String> combined = Stream.of(s1,s2).flatMap(s->s).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Combined: " + combined + ", Time(ms): " + (end-start)/1_000_000);
}

```

```

static void mapVsFlatMap12() {
    System.out.println("\nMapVsFlatMap12: Combined nested collections + Optional
flatten");
    List<List<Optional<String>>> data = Arrays.asList(
        Arrays.asList(Optional.of("x"), Optional.empty()),
        Arrays.asList(Optional.of("y"))
    );
    long start = System.nanoTime();
    List<String> flat =
data.stream().flatMap(List::stream).flatMap(Optional::stream).collect(Collectors.toList()
);
    long end = System.nanoTime();
    System.out.println("Flattened result: " + flat + ", Time(ms): " + (end-
start)/1_000_000);
}

```